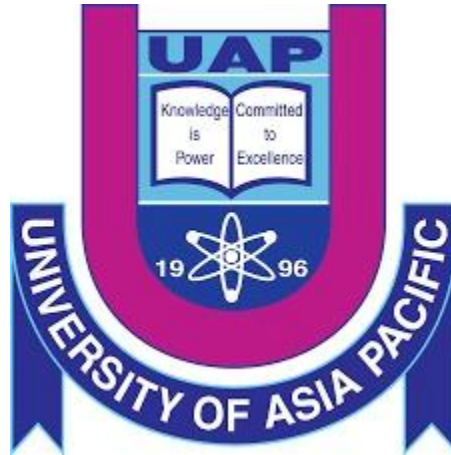




# UNIVERSITY OF ASIA PACIFIC

## Department of Computer Science and Engineering (CSE)

**Course Title:** Compiler Design Lab

**Course Code:** CSE-430

**Project Name:** Mini Compiler Design

Submitted By:

Name: Abdullah Al Musabbir

Reg. No: 22101039

Sec: A

Submitted To:

MD. SHAFAYATUL HAQUE

Lecturer

Department of CSE

University of Asia Pacific

## Contents

|                                                     |    |
|-----------------------------------------------------|----|
| 1. Introduction .....                               | 5  |
| 1.1 Problem Statement.....                          | 5  |
| 1.2 Project Motivation .....                        | 5  |
| 1.3 Background .....                                | 5  |
| 1.3.1 What is a Compiler? .....                     | 5  |
| 1.3.2 Tools Used.....                               | 6  |
| 1.3.3 The C Language Subset Handled.....            | 6  |
| 2. System Design and Methodology .....              | 7  |
| 2.1 System Overview / Architecture .....            | 7  |
| 2.2 File Structure.....                             | 8  |
| 2.3 Data Structures .....                           | 8  |
| 2.3.1 Token Structure.....                          | 8  |
| 2.3.2 Symbol Table Entry .....                      | 9  |
| 2.3.3 Three Address Code (TAC) .....                | 9  |
| 2.3.4 AST Node.....                                 | 9  |
| 2.4 Grammar Rules Overview .....                    | 10 |
| 2.5 Algorithmic Details.....                        | 10 |
| 2.5.1 Lexical Analysis Algorithm.....               | 10 |
| 2.5.2 Parsing Algorithm.....                        | 11 |
| 2.5.3 TAC Generation Algorithm .....                | 11 |
| 2.5.4 Optimization Algorithms .....                 | 11 |
| 2.5.5 Assembly Generation Algorithm.....            | 12 |
| 2.6 Error Handling Design .....                     | 12 |
| 2.6.1 Error Counter .....                           | 12 |
| 2.6.2 Error Categories by Phase .....               | 13 |
| 2.6.3 Error Recovery Strategy .....                 | 13 |
| 3. Implementation .....                             | 13 |
| 3.1 Module Breakdown.....                           | 13 |
| 3.2 Module 1: compiler.h — Shared Definitions ..... | 13 |
| 3.3 Module 2: Lexer (lexer.l) .....                 | 14 |
| Comment Handling: .....                             | 14 |
| Keyword Recognition: .....                          | 14 |

|                                                                  |    |
|------------------------------------------------------------------|----|
| Identifier & Number Patterns: .....                              | 15 |
| Operator Recognition (sample): .....                             | 16 |
| 3.4 Module 3: Parser (parser.y) .....                            | 16 |
| 3.4.1 Variable Declaration Handling .....                        | 16 |
| 3.4.2 Function Declaration .....                                 | 16 |
| 3.4.3 Expression with TAC Generation.....                        | 17 |
| 3.4.4 If-Else with Labels.....                                   | 17 |
| 3.5 Module 4: Core Engine (main.c) .....                         | 18 |
| 3.5.1 Symbol Table Operations .....                              | 18 |
| 3.5.2 Temporary and Label Generation .....                       | 18 |
| 3.5.3 AST Node Creation .....                                    | 19 |
| 3.5.4 AST Tree Printing.....                                     | 19 |
| 3.5.5 TAC Optimization .....                                     | 19 |
| 3.5.6 Assembly Generation (Sample).....                          | 20 |
| 3.6 Module 5: Test Input (test.c) .....                          | 21 |
| 3.7 Build & Execution .....                                      | 21 |
| 3.8 Error Handling Implementation .....                          | 21 |
| 3.8.1 Lexical Error Handling (lexer.l) .....                     | 21 |
| 3.8.2 Syntax Error Handling (parser.y) .....                     | 22 |
| 3.8.3 Semantic Error Handling (parser.y — grammar actions) ..... | 22 |
| 3.8.4 Semantic Error Detection in Step 3 Output .....            | 24 |
| 3.8.5 Error Handling Summary Table .....                         | 25 |
| 4. Results and Output .....                                      | 26 |
| 4.1 Compilation Pipeline Output .....                            | 26 |
| Step 1: Token Table.....                                         | 26 |
| Step 2: Symbol Table.....                                        | 26 |
| Step 3: Syntax & Semantic Analysis .....                         | 27 |
| Step 4: Abstract Syntax Tree .....                               | 27 |
| Step 5: Three Address Code (TAC) .....                           | 28 |
| Step 6: Optimized TAC .....                                      | 29 |
| Step 7: Assembly Code Generation.....                            | 29 |
| 4.2 Error Detection Output Examples.....                         | 30 |
| 4.2.1 Scenario 1: Clean Input (test.c — No Errors) .....         | 30 |

|                                                              |    |
|--------------------------------------------------------------|----|
| 4.2.2 Scenario 2: Lexical Error — Unknown Character .....    | 30 |
| 4.2.3 Scenario 3: Semantic Error — Undeclared Variable ..... | 31 |
| 4.2.4 Scenario 4: Semantic Error — Redeclared Variable ..... | 31 |
| 5. Conclusion and Future Scope.....                          | 32 |
| 5.1 Conclusion .....                                         | 32 |
| 5.2 Future Scope .....                                       | 32 |
| 5.2.1 Language Feature Extensions .....                      | 32 |
| 5.2.2 Compiler Improvements .....                            | 33 |
| 5.2.3 Tooling Improvements .....                             | 33 |
| 6. References .....                                          | 33 |
| 7. Appendices.....                                           | 34 |
| Appendix A: Full Source Code (GitHub Link) .....             | 34 |
| Appendix B: Sample Input File (test.c) .....                 | 34 |
| Appendix C: Error Types and Handling .....                   | 36 |

# 1. Introduction

## 1.1 Problem Statement

Writing and understanding a compiler from scratch is one of the most challenging yet fundamental tasks in computer science. A compiler translates source code written in a high-level programming language into a lower-level representation ultimately into machine code or assembly while performing lexical analysis, syntax checking, semantic validation, intermediate code generation, optimization, and final code generation.

The problem addressed in this project is: Can we build a working mini compiler for the C language that demonstrates all major phases of compilation in a structured, step-by-step manner?

The mini compiler must be capable of:

- Reading a C source file (test.c)
- Tokenizing the source into a structured token table (Lexical Analysis)
- Building and maintaining a Symbol Table
- Performing Syntax and Semantic Analysis
- Constructing an Abstract Syntax Tree (AST)
- Generating Three Address Code (TAC)
- Optimizing the TAC (Copy Propagation + Constant Folding)
- Generating x86 Assembly Code (NASM style)

## 1.2 Project Motivation

The motivation behind building this mini compiler comes from the desire to deeply understand how high-level C code is transformed step-by-step into machine understandable instructions. Most programmers use compilers like GCC or Clang without knowing the internal workings. By implementing a compiler from scratch using standard tools (Flex/Lex for lexing, Bison/Yacc for parsing), this project bridges the gap between theoretical compiler design and practical implementation.

Additionally, this project serves as a comprehensive demonstration of:

- How tokens are classified and stored
- How a grammar is defined and parsed
- How scope, type, and symbol information are tracked
- How intermediate representations (TAC) are generated and optimized
- How assembly instructions are mapped from high-level operations

## 1.3 Background

### 1.3.1 What is a Compiler?

A compiler is a program that translates source code written in one language (typically a high-level language) into another language (typically low-level machine code or assembly). The compilation process is divided into several well-defined phases:

| Phase                        | Description                           |
|------------------------------|---------------------------------------|
| Lexical Analysis             | Breaks source into tokens             |
| Syntax Analysis              | Parses tokens into a tree structure   |
| Semantic Analysis            | Checks meaning and type correctness   |
| Intermediate Code Generation | Produces TAC or similar IR            |
| Code Optimization            | Improves efficiency of the IR         |
| Code Generation              | Produces target machine/assembly code |

### 1.3.2 Tools Used

| Tool         | Purpose                           |
|--------------|-----------------------------------|
| Flex (Lex)   | Lexical Analyzer Generator        |
| Bison (Yacc) | Parser Generator (LALR(1))        |
| C Language   | Implementation Language           |
| GCC          | Compiler for building the project |
| NASM         | Target assembly style (x86)       |

### 1.3.3 The C Language Subset Handled

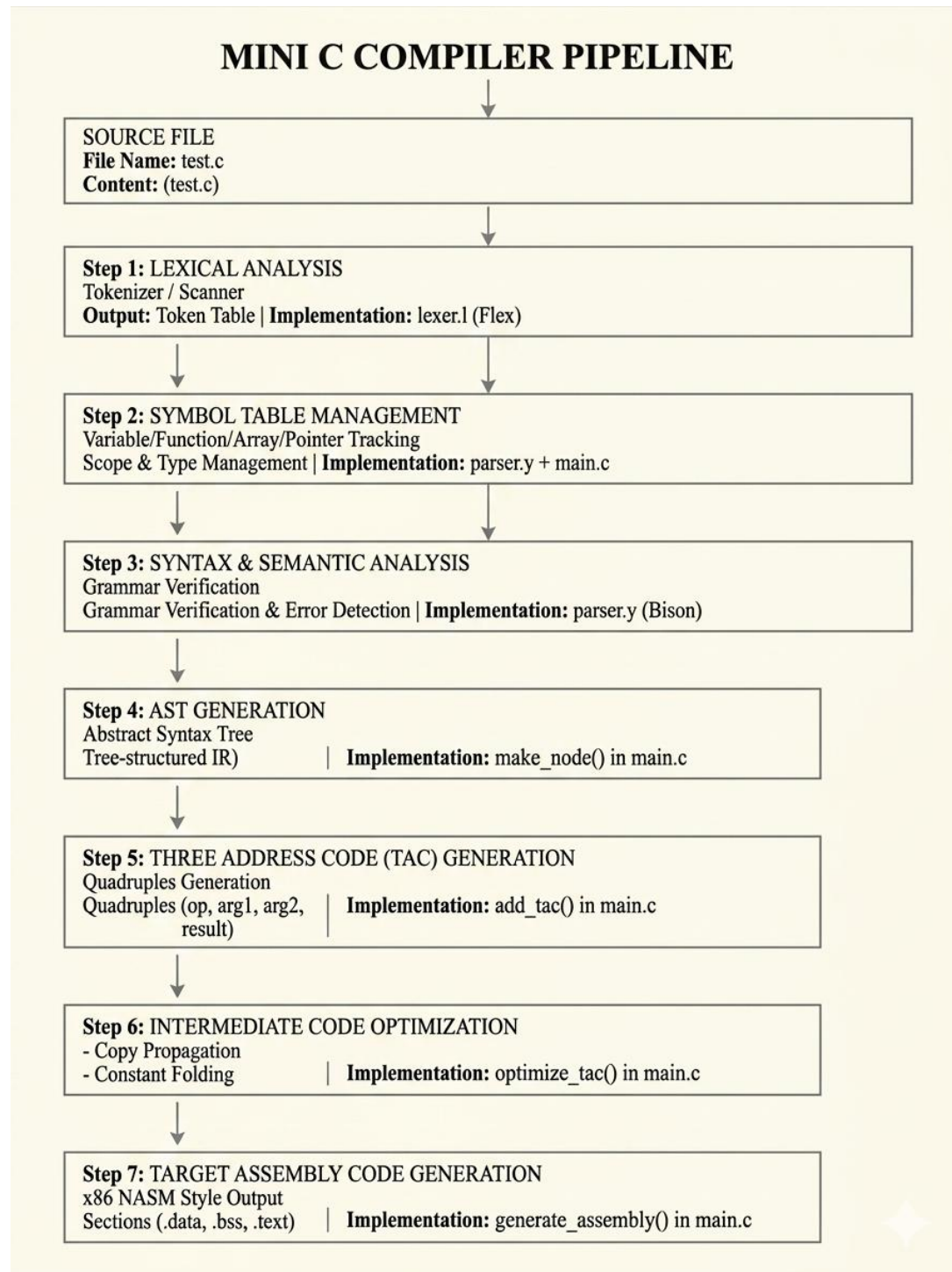
This mini compiler supports a significant subset of C, including:

- Data types: int, float, char, void
- Variables, arrays, and pointers
- Structs
- Functions with parameters and return values
- Control flow: if, if-else, while, for, switch-case
- Operators: arithmetic, relational, logical, unary
- printf statements
- Comments (single-line and multi-line)

## 2. System Design and Methodology

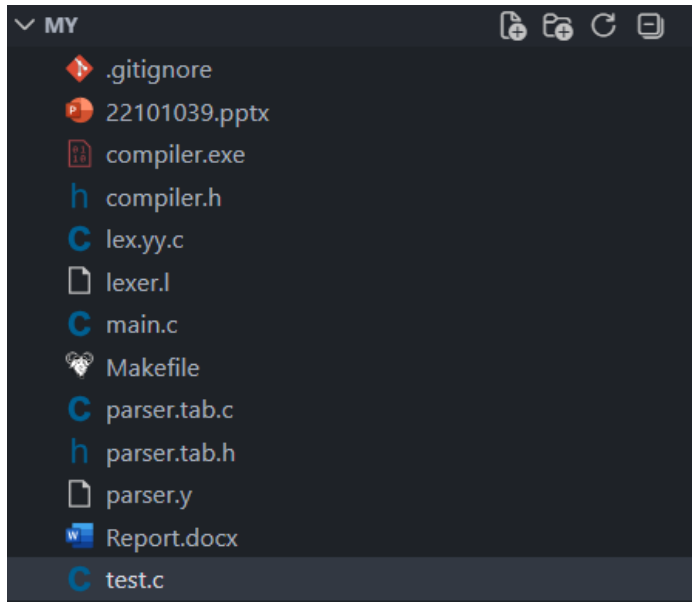
### 2.1 System Overview / Architecture

The mini compiler is designed as a pipeline architecture where each phase receives input from the previous phase and produces output for the next. The overall architecture is shown below:



## 2.2 File Structure

The project consists of the following source files:



## 2.3 Data Structures

### 2.3.1 Token Structure

```
typedef struct {  
    char name[MAX_NAME];  
    char value[MAX_NAME];  
    TokenCategory category;  
    int line;  
} Token;
```

Token Categories:

| Enum            | Category                                    |
|-----------------|---------------------------------------------|
| TOK_KEYWORD     | Reserved words: int, float, if, while, etc. |
| TOK_IDENTIFIER  | Variable/function names                     |
| TOK_NUMBER      | Numeric literals                            |
| TOK_OPERATOR    | +, -, *, /, ==, !=, etc.                    |
| TOK_PUNCTUATION | ;, ,, (, ), {, }, [, ]                      |
| TOK_STRING      | String literals                             |
| TOK_UNKNOWN     | Unrecognized characters                     |



### 2.3.2 Symbol Table Entry

```
typedef struct {  
    char name[MAX_NAME];  
    char type[MAX_NAME];  
    char scope[MAX_NAME];  
    int line;  
    int initialized;  
    int is_array;  
    int array_size;  
    int is_pointer;  
    int is_function;  
    int param_count;  
    char param_types[MAX_PARAMS][MAX_NAME];  
} Symbol;
```

### 2.3.3 Three Address Code (TAC)

```
typedef struct {  
    char op[MAX_NAME];  
    char arg1[MAX_NAME];  
    char arg2[MAX_NAME];  
    char result[MAX_NAME];  
} TAC;
```

TAC follows the quadruple form: (op, arg1, arg2, result)

Examples:

| op       | arg1 | arg2 | result |
|----------|------|------|--------|
| +        | x    | y    | t0     |
| =        | t0   | ''   | z      |
| IF_FALSE | t1   | ''   | L2     |
| CALL     | add  | ''   | t3     |

### 2.3.4 AST Node

```
typedef struct ASTNode {  
    char type[MAX_NAME];  
    char value[MAX_NAME];  
    struct ASTNode *left;  
    struct ASTNode *right;  
    struct ASTNode *extra;  
    struct ASTNode *extra2;  
} ASTNode;
```

## 2.4 Grammar Rules Overview

The parser (parser.y) implements an LALR(1) grammar using Bison. Key grammar productions include:

```
program    → decl_list
decl_list  → decl_list decl | decl
decl       → func_decl | var_decl | struct_decl

var_decl   → type_spec ID SEMI
            | type_spec ID ASSIGN expr SEMI
            | type_spec ID LBRACKET NUMBER RBRACKET SEMI
            | type_spec MUL ID SEMI

func_decl   → type_spec ID LPAREN RPAREN func_body
            | type_spec ID LPAREN param_list RPAREN func_body

stmt        → var_decl | assign_stmt | if_stmt | while_stmt
            | for_stmt | switch_stmt | return_stmt
            | printf_stmt | func_call_stmt | BREAK SEMI

expr        → expr PLUS term | expr MINUS term
            | expr AND expr | expr OR expr
            | expr LT term | expr GT term
            | expr EQ term | expr NEQ term
            | term

term        → term MUL factor | term DIV factor | factor

factor      → NUMBER | ID | ID[expr] | ID(args)
            | *ID | &ID | (expr) | -factor | !factor
```

Operator Precedence (low to high):

```
ASSIGN
OR (||)
AND (&&)
EQ, NEQ
LT, GT, LEQ, GEQ
PLUS, MINUS
MUL, DIV
NOT, AMP (unary, right-associative)
```

## 2.5 Algorithmic Details

### 2.5.1 Lexical Analysis Algorithm

The Flex lexer (lexer.l) uses Deterministic Finite Automaton (DFA) based pattern matching:

1. Read source character by character
2. Match against regex patterns (longest match wins)

3. For each match:
  - a. Call `add_token(name, value, category, line)`
  - b. Return appropriate token type to parser
4. Track line numbers (`yyline++`)
5. Skip whitespace and comments
6. Report unknown characters as LEX ERROR

### 2.5.2 Parsing Algorithm

Bison generates an LALR(1) parser using shift-reduce parsing:

1. Receive tokens from lexer (`yylex()`)
2. Maintain parsing stack
3. On each token:
  - a. SHIFT: push token onto stack
  - b. REDUCE: apply grammar rule, build AST node
4. On grammar rule match:
  - a. Call `add_symbol_ex()` for declarations
  - b. Call `add_tac()` for code generation
  - c. Call `make_node()` for AST construction
5. On syntax error: call `yyerror()`, increment `error_count`

### 2.5.3 TAC Generation Algorithm

TAC is generated during parsing (syntax-directed translation):

**For each expression node:**

```
t = new_temp()    // generate temporary: t0, t1, t2...
add_tac(op, arg1, arg2, t)
return t as the value of the node
```

**For control flow:**

```
L = new_label()    // generate label: L0, L1, L2...
add_tac("IF_FALSE", condition, "", L)
add_tac("LABEL", L, "", "")
```

### 2.5.4 Optimization Algorithms

Copy Propagation:

**For each TAC instruction i where `op == "="` and result is temp:**

```
from = result
to = arg1
For each subsequent instruction j:
  Replace all uses of 'from' with 'to'
  Stop if 'from' is assigned again
Mark instruction i as NOP
```

Constant Folding:

**For each TAC instruction where `arg1` and `arg2` are both numbers:**

Evaluate the expression at compile time:

"+"  $\rightarrow n1 + n2$

"-"  $\rightarrow n1 - n2$

"\*"  $\rightarrow n1 * n2$

"/"  $\rightarrow n1 / n2$  (if  $n2 \neq 0$ )

Replace the instruction with: "=" constant result

### 2.5.5 Assembly Generation Algorithm

#### 1. Emit section .data

For each non-function symbol:

Emit: name dd 0 (int), dq 0.0 (float), db 0 (char)

For arrays: times N dd 0

#### 2. Emit section .bss

For each temporary variable tN: resd 1

#### 3. Emit section .text

For each TAC instruction:

FUNC\_START  $\rightarrow$  label: + push ebp + mov ebp, esp

FUNC\_END  $\rightarrow$  pop ebp + ret

"="  $\rightarrow$  mov eax, src + mov [dst], eax

"+"  $\rightarrow$  mov eax, a + add eax, b + mov [res], eax

"-"  $\rightarrow$  mov eax, a + sub eax, b + mov [res], eax

"\*"  $\rightarrow$  mov eax, a + imul eax, b + mov [res], eax

"/"  $\rightarrow$  mov eax, a + cdq + idiv b + mov [res], eax

IF\_FALSE  $\rightarrow$  mov eax, cond + cmp eax, 0 + je label

GOTO  $\rightarrow$  jmp label

CALL  $\rightarrow$  call funcname + mov [res], eax

RETURN  $\rightarrow$  mov eax, val + pop ebp + ret

## 2.6 Error Handling Design

Error handling is one of the most critical aspects of any compiler. This mini compiler implements multi-level error detection and reporting across all compilation phases. The design goal is to:

- Detect errors as early as possible
- Report meaningful error messages with line numbers
- Continue compilation after errors (error recovery)
- Count total errors for final summary

### 2.6.1 Error Counter

A global error counter is maintained throughout all phases:

**extern int error\_count;**

This counter is incremented whenever any phase detects an error or warning. At the end of compilation, the total is reported:

**Total Errors : 3**

### 2.6.2 Error Categories by Phase

| Phase             | Error Type                   | Prefix in Output             |
|-------------------|------------------------------|------------------------------|
| Lexical Analysis  | Unknown/invalid character    | [LEX ERROR]                  |
| Syntax Analysis   | Grammar rule violation       | [PARSE ERROR] or [SYN ERROR] |
| Semantic Analysis | Undeclared variable/function | [WARN]                       |
| Semantic Analysis | Redeclared variable          | [WARN]                       |

### 2.6.3 Error Recovery Strategy

The compiler uses panic-mode error recovery at the statement level:

When a syntax error occurs:

1. Call `yyerror()` → print error message, increment `error_count`
2. Bison activates error recovery
3. Parser discards tokens until it finds a SEMI (;)
4. Call `yyerrok` → resume normal parsing from next statement
5. Compilation continues — more errors can be detected

This ensures that a single error does not stop the entire compilation — the compiler will continue to find and report subsequent errors.

## 3. Implementation

### 3.1 Module Breakdown

The project is divided into 5 major modules:

| Module               | File                    | Responsibility                                                      |
|----------------------|-------------------------|---------------------------------------------------------------------|
| Header & Definitions | <code>compiler.h</code> | All shared struct/type definitions                                  |
| Lexer                | <code>lexer.l</code>    | Tokenization (Flex)                                                 |
| Parser               | <code>parser.y</code>   | Grammar, AST, TAC generation (Bison)                                |
| Core Engine          | <code>main.c</code>     | Symbol table, printing, optimization, assembly, <code>main()</code> |
| Test Input           | <code>test.c</code>     | Sample C program for compilation                                    |

### 3.2 Module 1: `compiler.h` — Shared Definitions

This header file acts as the central hub connecting all modules. It defines:

- Constants: `MAX_TOKENS` (2000), `MAX_SYMBOLS` (200), `MAX_TAC` (2000)

- Enums: TokenCategory
- Structs: Token, Symbol, TAC, ASTNode
- Global Arrays: token\_table[], symbol\_table[], tac\_list[]
- Function Declarations for all major routines

Key design choice: All global arrays are declared with extern in the header and defined once in main.c, avoiding multiple-definition linker errors.

```
typedef struct ASTNode {
    char type[MAX_NAME];
    char value[MAX_NAME];
    struct ASTNode *left;
    struct ASTNode *right;
    struct ASTNode *extra;
    struct ASTNode *extra2;
} ASTNode;
```

### 3.3 Module 2: Lexer (lexer.l)

The lexer is written using Flex and follows these rules:

Comment Handling:

```
"/*" {
    /* Skip multi-line comments, tracking newlines */
    int c;
    while(1) {
        c = input();
        if(c == EOF) break;
        if(c == '\n') yyline++;
        if(c == '*') {
            c = input();
            if(c == '/') break;
        }
    }
}
"//".* { /* Skip single-line comment */ }
```

Keyword Recognition:

```
"int"    { add_token("int",  yytext, TOK_KEYWORD, yyline); return INT;  }
"float"  { add_token("float", yytext, TOK_KEYWORD, yyline); return FLOAT; }
"char"   { add_token("char",  yytext, TOK_KEYWORD, yyline); return CHAR; }
"void"   { add_token("void",   yytext, TOK_KEYWORD, yyline); return VOID; }
"if"     { add_token("if",     yytext, TOK_KEYWORD, yyline); return IF;   }
"else"   { add_token("else",   yytext, TOK_KEYWORD, yyline); return ELSE; }
"while"  { add_token("while",  yytext, TOK_KEYWORD, yyline); return WHILE; }
"for"    { add_token("for",    yytext, TOK_KEYWORD, yyline); return FOR;   }
"return" { add_token("return", yytext, TOK_KEYWORD, yyline); return RETURN; }
"printf" { add_token("printf", yytext, TOK_KEYWORD, yyline); return PRINTF; }
```

```

"struct" { add_token("struct", yytext, TOK_KEYWORD, yyline); return STRUCT; }
"switch" { add_token("switch", yytext, TOK_KEYWORD, yyline); return SWITCH; }
"case" { add_token("case", yytext, TOK_KEYWORD, yyline); return CASE; }
"break" { add_token("break", yytext, TOK_KEYWORD, yyline); return BREAK; }
"default" { add_token("default", yytext, TOK_KEYWORD, yyline); return DEFAULT; }

```

```

{NUMBER} { add_token("NUMBER", yytext, TOK_NUMBER, yyline);
           yyval.str = strdup(yytext); return NUMBER; }

```

```

{ID} { add_token("ID", yytext, TOK_IDENTIFIER, yyline);
       yyval.str = strdup(yytext); return ID; }

```

```

{STRING} { add_token("STRING", yytext, TOK_STRING, yyline);
           yyval.str = strdup(yytext); return STRING_LIT; }

```

```

"+" { add_token("+", yytext, TOK_OPERATOR, yyline); return PLUS; }
"-" { add_token("-", yytext, TOK_OPERATOR, yyline); return MINUS; }
"*" { add_token("*", yytext, TOK_OPERATOR, yyline); return MUL; }
"/" { add_token("/", yytext, TOK_OPERATOR, yyline); return DIV; }
"=" { add_token "=", yytext, TOK_OPERATOR, yyline); return ASSIGN; }
"==" { add_token("==", yytext, TOK_OPERATOR, yyline); return EQ; }
"!=" { add_token("!= ", yytext, TOK_OPERATOR, yyline); return NEQ; }
"<" { add_token("<", yytext, TOK_OPERATOR, yyline); return LT; }
">" { add_token(">", yytext, TOK_OPERATOR, yyline); return GT; }
"<=" { add_token("<=", yytext, TOK_OPERATOR, yyline); return LEQ; }
">=" { add_token(">=", yytext, TOK_OPERATOR, yyline); return GEQ; }
"&&" { add_token("&&", yytext, TOK_OPERATOR, yyline); return AND; }
"||" { add_token("||", yytext, TOK_OPERATOR, yyline); return OR; }
"!" { add_token("!", yytext, TOK_OPERATOR, yyline); return NOT; }
"&" { add_token("&", yytext, TOK_OPERATOR, yyline); return AMP; }
"->" { add_token("->", yytext, TOK_OPERATOR, yyline); return ARROW; }
"++" { add_token("++", yytext, TOK_OPERATOR, yyline); return INC; }
"--" { add_token("--", yytext, TOK_OPERATOR, yyline); return DEC; }

```

```

";" { add_token(";", yytext, TOK_PUNCTUATION, yyline); return SEMI; }
"," { add_token(",", yytext, TOK_PUNCTUATION, yyline); return COMMA; }
"(" { add_token("(", yytext, TOK_PUNCTUATION, yyline); return LPAREN; }
")" { add_token(")", yytext, TOK_PUNCTUATION, yyline); return RPAREN; }
"{" { add_token("{", yytext, TOK_PUNCTUATION, yyline); return LBRACE; }
"}" { add_token("}", yytext, TOK_PUNCTUATION, yyline); return RBRACE; }
"[" { add_token("[", yytext, TOK_PUNCTUATION, yyline); return LBRACKET; }
"]" { add_token("]", yytext, TOK_PUNCTUATION, yyline); return RBRACKET; }
"." { add_token(".", yytext, TOK_PUNCTUATION, yyline); return DOT; }
":" { add_token(":", yytext, TOK_PUNCTUATION, yyline); return COLON; }

```

Identifier & Number Patterns:

**DIGIT** [0-9]

**LETTER** [a-zA-Z\_]

```
ID    {LETTER}({LETTER}|{DIGIT})*
NUMBER {DIGIT}+(\.{DIGIT}+)?
```

```
{ID}  { add_token("ID", yytext, TOK_IDENTIFIER, yyline);
        yyval.str = strdup(yytext); return ID; }
{NUMBER} { add_token("NUMBER", yytext, TOK_NUMBER, yyline);
        yyval.str = strdup(yytext); return NUMBER; }
```

Operator Recognition (sample):

```
"==" { add_token("==", yytext, TOK_OPERATOR, yyline); return EQ; }
"!=" { add_token("!", yytext, TOK_OPERATOR, yyline); return NEQ; }
"++" { add_token("++", yytext, TOK_OPERATOR, yyline); return INC; }
"->" { add_token("->", yytext, TOK_OPERATOR, yyline); return ARROW; }
```

Key Feature: Two-character operators (==, !=, <=, >=, &&, ||, ++, --, ->) are matched before single-character ones, ensuring correct tokenization.

### 3.4 Module 3: Parser (parser.y)

The parser is written using Bison (YACC) and implements LALR(1) parsing.

#### 3.4.1 Variable Declaration Handling

```
var_decl
/* int x; */
: type_spec ID SEMI
{
    if(find_symbol($2) == NULL)
        add_symbol_ex($2, $1, current_scope, yyline, 0,0,0,0);
    else {
        printf("[WARN] Line %d: '%s' redeclared\n", yyline, $2);
        error_count++;
    }
    add_tac("DECL", $1, "", $2);
    $$ = make_node("VAR_DECL", $2, NULL, NULL);
}
/* int arr[10]; */
| type_spec ID LBRACKET NUMBER RBRACKET SEMI
{
    int sz = atoi($4);
    add_symbol_ex($2, $1, current_scope, yyline, 1, sz, 0,0,0);
    add_tac("ARRAY_DECL", $1, szbuf, $2);
    $$ = make_node("ARRAY_DECL", $2, NULL, NULL);
}
```

#### 3.4.2 Function Declaration

```
func_decl
: type_spec ID LPAREN param_list RPAREN
{

```



```

    strcpy(current_scope, $2);
    strcpy(current_func, $2);
    add_symbol_ex($2, $1, "global", yyline, 0,0,0,1, param_count_buf);
    add_tac("FUNC_START", $2, "", "");
}
func_body
{
    $$ = make_node("FUNC_DECL", $2, $7, NULL);
    add_tac("FUNC_END", $2, "", "");
    strcpy(current_scope, "global");
    param_count_buf = 0;
}

```

### 3.4.3 Expression with TAC Generation

expr

```

: expr PLUS term
{
    char* t = new_temp();
    add_tac("+", $1->value, $3->value, t);
    $$ = make_node("ADD", t, $1, $3);
    strcpy($$->value, t);
}
| expr AND expr /* Short-circuit evaluation */
{
    char* t = new_temp();
    char lf[MAX_NAME], le[MAX_NAME];
    strcpy(lf, new_label());
    strcpy(le, new_label());
    add_tac("IF_FALSE", $1->value, "", lf);
    add_tac("IF_FALSE", $3->value, "", lf);
    add_tac("=", "1", "", t);
    add_tac("GOTO", "", "", le);
    add_tac("LABEL", lf, "", "");
    add_tac("=", "0", "", t);
    add_tac("LABEL", le, "", "");
    $$ = make_node("AND", t, $1, $3);
}

```

### 3.4.4 If-Else with Labels

if\_stmt

```

: IF LPAREN expr RPAREN LBRACE stmt_list RBRACE
  ELSE LBRACE stmt_list RBRACE
{
    char lbl_else[MAX_NAME], lbl_end[MAX_NAME];
    strcpy(lbl_else, new_label());
    strcpy(lbl_end, new_label());
    add_tac("IF_FALSE", $3->value, "", lbl_else);
    add_tac("GOTO", "", "", lbl_end);
    add_tac("LABEL", lbl_else, "", "");
}

```

```

    add_tac("LABEL", lbl_end, "", "");
    $$ = make_node("IF_ELSE", "", $3,
        make_node("BRANCHES", "", $6, $10));
}

```

### 3.5 Module 4: Core Engine (main.c)

#### 3.5.1 Symbol Table Operations

```

// Add a symbol with extended attributes
void add_symbol_ex(const char* name, const char* type,
    const char* scope, int line,
    int is_array, int array_size,
    int is_pointer, int is_function, int param_count) {
    if(symbol_count >= MAX_SYMBOLS) return;
    strncpy(symbol_table[symbol_count].name, name, MAX_NAME-1);
    strncpy(symbol_table[symbol_count].type, type, MAX_NAME-1);
    strncpy(symbol_table[symbol_count].scope, scope, MAX_NAME-1);
    symbol_table[symbol_count].line = line;
    symbol_table[symbol_count].is_array = is_array;
    symbol_table[symbol_count].array_size = array_size;
    symbol_table[symbol_count].is_pointer = is_pointer;
    symbol_table[symbol_count].is_function = is_function;
    symbol_table[symbol_count].param_count = param_count;
    symbol_count++;
}

// Find a symbol by name (linear search)
Symbol* find_symbol(const char* name) {
    for(int i = 0; i < symbol_count; i++)
        if(strcmp(symbol_table[i].name, name) == 0)
            return &symbol_table[i];
    return NULL;
}

```

#### 3.5.2 Temporary and Label Generation

```

// Generates: t0, t1, t2, ...
char* new_temp() {
    sprintf(temp_buf, "t%d", temp_count++);
    return temp_buf;
}

// Generates: L0, L1, L2, ...
char* new_label() {
    sprintf(label_buf, "L%d", label_count++);
    return label_buf;
}

```

### 3.5.3 AST Node Creation

```
ASTNode* make_node(const char* type, const char* value,
    ASTNode* left, ASTNode* right) {
    ASTNode* n = (ASTNode*)malloc(sizeof(ASTNode));
    strncpy(n->type, type, MAX_NAME-1);
    strncpy(n->value, value, MAX_NAME-1);
    n->left = left;
    n->right = right;
    n->extra = NULL;
    n->extra2 = NULL;
    return n;
}
```

### 3.5.4 AST Tree Printing

The AST is printed as a text-based tree using recursive DFS:

```
static void print_tree(ASTNode* node, char* prefix, int is_last) {
    if(!node) return;
    printf("%s%s", prefix, is_last ? "L-- " : "|-- ");
    char label[MAX_NAME*2];
    node_label(node, label);
    printf("%s\n", label);

    ASTNode* ch[64]; int cnt = 0;
    collect_children(node, ch, &cnt);
    char new_pre[512];
    sprintf(new_pre, "%s%s", prefix, is_last ? "  " : "| ");
    for(int i = 0; i < cnt; i++)
        print_tree(ch[i], new_pre, i == cnt-1);
}
```

Intermediate nodes (PROGRAM, DECL\_LIST, STMT\_LIST) are automatically flattened by collect\_children() to produce a clean, readable tree.

### 3.5.5 TAC Optimization

```
void optimize_tac() {
    /* Pass 1: Copy Propagation */
    for(int i = 0; i < tac_count; i++) {
        if(strcmp(tac_list[i].op, "=") == 0 &&
            tac_list[i].result[0] == 't') {
            char* from = tac_list[i].result;
            char* to = tac_list[i].arg1;
            for(int j = i+1; j < tac_count; j++) {
                if(strcmp(tac_list[j].arg1, from) == 0)
                    strcpy(tac_list[j].arg1, to);
                if(strcmp(tac_list[j].arg2, from) == 0)
                    strcpy(tac_list[j].arg2, to);
                if(strcmp(tac_list[j].result, from) == 0) break;
            }
        }
    }
}
```

```

        strcpy(tac_list[i].op, "NOP");
    }
}

/* Pass 2: Constant Folding */
for(int i = 0; i < tac_count; i++) {
    TAC* t = &tac_list[i];
    if(!lis_number(t->arg1) || !lis_number(t->arg2)) continue;
    double n1 = atof(t->arg1), n2 = atof(t->arg2), res = 0;
    int ok = 1;
    if (strcmp(t->op, "+") == 0) res = n1 + n2;
    else if (strcmp(t->op, "-") == 0) res = n1 - n2;
    else if (strcmp(t->op, "*") == 0) res = n1 * n2;
    else if (strcmp(t->op, "/") == 0 && n2 != 0) res = n1 / n2;
    else ok = 0;
    if(ok) {
        char buf[MAX_NAME];
        if(res == (int)res) sprintf(buf, "%d", (int)res);
        else sprintf(buf, "%f", res);
        strcpy(t->op, "=");
        strcpy(t->arg1, buf);
        strcpy(t->arg2, "");
    }
}
}

```

### 3.5.6 Assembly Generation (Sample)

```

// Assignment: x = y
if(strcmp(t->op, "=") == 0) {
    if(is_number(t->arg1)) printf(" mov eax, %s\n", t->arg1);
    else printf(" mov eax, [%s]\n", t->arg1);
    printf(" mov [%s], eax\n", t->result);
}

```

```

// Addition: t = a + b
if(strcmp(t->op, "+") == 0) {
    printf(" mov eax, [%s]\n", t->arg1);
    printf(" add eax, [%s]\n", t->arg2);
    printf(" mov [%s], eax\n", t->result);
}

```

```

// Conditional jump: IF_FALSE cond goto L
if(strcmp(t->op, "IF_FALSE") == 0) {
    printf(" mov eax, [%s]\n", t->arg1);
    printf(" cmp eax, 0\n");
    printf(" je %s\n", t->result);
}

```

### 3.6 Module 5: Test Input (test.c)

The test file exercises all major features of the compiler:

```
struct Point { int x; int y; };
int arr[5];
int *ptr;
int add(int a, int b) { ... }
int square(int n) { ... }
int main() {
    int x = 10; int y = 20;
    arr[0] = 5;
    ptr = &x; z = *ptr;
    z = add(x, y);
    c = a && b; c = a || b;
    if (x > y) { ... } else { ... }
    while (x > 0) { x = x - 1; }
    for (int i = 0; i < 5; i++) { ... }
    switch (z) { case 1: ... case 2: ... default: ... }
    printf("Result: %d", z);
    return z;
}
```

### 3.7 Build & Execution

Compilation Commands:

```
flex lexer.l
bison -d parser.y
gcc -w -o compiler lex.yy.c parser.tab.c main.c
./compiler
```

### 3.8 Error Handling Implementation

#### 3.8.1 Lexical Error Handling (lexer.l)

The lexer handles unknown characters using a catch-all rule at the bottom of the rules section. This rule matches any character that did not match any other pattern:

```
.{
    /* Catch-all rule: matches any unrecognized character */
    add_token("UNKNOWN", yytext, TOK_UNKNOWN, yyline);
    printf("[LEX ERROR] Unknown char '%s' at line %d\n",
        yytext, yyline);
    error_count++;
}
```

What happens:

- The unknown character is still added to the token table with category TOK\_UNKNOWN
- An error message is printed immediately with the character and line number
- error\_count is incremented
- The lexer continues scanning — it does not abort

### 3.8.2 Syntax Error Handling (parser.y)

Bison automatically calls `yyerror()` when the parser encounters a token that does not fit any grammar rule. The implementation:

```
void yyerror(const char* msg) {  
    printf("[PARSE ERROR] Line %d: %s\n", yyline, msg);  
    error_count++;  
}
```

Additionally, the grammar includes an explicit error recovery rule in the statement list:

```
stmt  
: var_decl    { $$ = $1; }  
| assign_stmt { $$ = $1; }  
| if_stmt     { $$ = $1; }  
/* ... other valid statements ... */  
  
/* ERROR RECOVERY RULE */  
| error SEMI  
{  
    printf("[SYN ERROR] Line %d: Invalid statement\n", yyline);  
    error_count++;  
    $$ = make_node("ERROR_STMT", "", NULL, NULL);  
    yyerrok; /* Tell Bison error has been handled */  
}  
;
```

How error recovery works step by step:

**Step 1:** Parser sees unexpected token (e.g., missing semicolon)

→ `yyerror()` is called: "[PARSE ERROR] Line N: ..."

**Step 2:** Bison enters error recovery mode

→ Starts discarding tokens from the input

**Step 3:** The special 'error' token in the rule matches

**Step 4:** Parser waits for SEMI (semicolon) as synchronization point

**Step 5:** On finding SEMI:

→ Creates `ERROR_STMT` node in AST

→ Calls `yyerrok` to clear error state

**Step 6:** Parsing resumes normally from the next statement

### 3.8.3 Semantic Error Handling (parser.y — grammar actions)

Semantic checks are embedded directly inside grammar action blocks. There are three types of semantic errors detected:

**Type 1:** Variable Redclaration

Detected during variable declaration parsing:

```

var_decl
: type_spec ID SEMI
{
    if(find_symbol($2) == NULL) {
        /* OK: First time declaring this variable */
        add_symbol_ex($2, $1, current_scope, yyline, 0,0,0,0,0);
    } else {
        /* ERROR: Variable already exists in symbol table */
        printf("[WARN] Line %d: '%s' redeclared\n", yyline, $2);
        error_count++;
    }
    add_tac("DECL", $1, "", $2);
    $$ = make_node("VAR_DECL", $2, NULL, NULL);
}

```

#### Type 2: Use of Undeclared Variable

Detected when an identifier appears in an expression:

```

factor
: ID
{
    /* Check if this identifier is in the symbol table */
    if(find_symbol($1) == NULL) {
        printf("[WARN] Line %d: '%s' undeclared\n", yyline, $1);
        error_count++;
    }
    $$ = make_node("ID", $1, NULL, NULL);
}

```

#### Type 3: Undeclared Variable in Assignment

When the left-hand side of an assignment is not declared, the compiler warns and auto-inserts a symbol to allow compilation to continue:

```

assign_stmt
: ID ASSIGN expr SEMI
{
    Symbol* s = find_symbol($1);
    if(s == NULL) {
        /* Variable not declared — report error but continue */
        printf("[WARN] Line %d: '%s' undeclared\n", yyline, $1);
        error_count++;
        /* Auto-insert with 'unknown' type to prevent cascade errors */
        add_symbol_ex($1, "unknown", current_scope,
            yyline, 0,0,0,0,0);
    } else {
        /* Mark as initialized */
        s->initialized = 1;
    }
    add_tac("=", $3->value, "", $1);
}

```

```

    $$ = make_node("ASSIGN", $1, $3, NULL);
}

```

#### Type 4: Undeclared Function Call

When a function is called but not declared in the symbol table:

```

func_call_stmt
: ID LPAREN arg_list RPAREN SEMI
{
    Symbol* s = find_symbol($1);
    if(s == NULL) {
        printf("[WARN] Line %d: function '%s' undeclared\n",
            yyline, $1);
        error_count++;
    }
    /* Continue generating TAC even for undeclared function */
    char* t = new_temp();
    add_tac("CALL", $1, "", t);
    $$ = make_node("FUNC_CALL", $1, NULL, NULL);
}

```

#### 3.8.4 Semantic Error Detection in Step 3 Output

In addition to real-time warnings during parsing, Step 3 (Syntax & Semantic Analysis) performs a post-parse scan of all tokens and cross-references identifiers against the final symbol table:

```

void print_token_stream() {
    print_header("STEP 3: SYNTAX & SEMANTIC ANALYSIS");

    int syn_err = 0, sem_err = 0;

    for(int i = 0; i < token_count; i++) {
        printf(" Token %-3d : %-14s | %-22s | Line: %d\n",
            i+1,
            category_name(token_table[i].category),
            token_table[i].value,
            token_table[i].line);

        /* Semantic check: is this identifier declared? */
        if(token_table[i].category == TOK_IDENTIFIER) {
            if(find_symbol(token_table[i].value) == NULL) {
                printf(" [!] SEMANTIC: '%s' undeclared at line %d\n",
                    token_table[i].value,
                    token_table[i].line);
                sem_err++;
            }
        }
    }

    /* Syntax check: is this an unknown token? */
}

```



```

    if(token_table[i].category == TOK_UNKNOWN) {
        printf(" [!] SYNTAX: unknown token '%s' at line %d\n",
            token_table[i].value,
            token_table[i].line);
        syn_err++;
    }
}

/* Final verdict */
printf(" Syntax Errors : %d\n", syn_err);
printf(" Semantic Errors: %d\n", sem_err);

if(syn_err == 0 && sem_err == 0)
    printf(" [OK] ANALYSIS PASSED\n");
else
    printf(" [FAIL] ANALYSIS FAILED\n");
}

```

3.8.5 Error Handling Summary Table

| Error                | Where Detected | Function/Rule        | Output                                  |
|----------------------|----------------|----------------------|-----------------------------------------|
| Unknown character    | lexer.l        | catch-all . rule     | [LEX ERROR] Unknown char 'X' at line N  |
| Grammar violation    | parser.y       | yyerror()            | [PARSE ERROR] Line N: syntax error      |
| Invalid statement    | parser.y       | error SEMI rule      | [SYN ERROR] Line N: Invalid statement   |
| Variable redeclared  | parser.y       | var_decl action      | [WARN] Line N: 'x' redeclared           |
| Variable undeclared  | parser.y       | factor / assign_stmt | [WARN] Line N: 'x' undeclared           |
| Function undeclared  | parser.y       | func_call_stmt       | [WARN] Line N: function 'f' undeclared  |
| Unknown token (post) | main.c         | print_token_stream() | [!] SYNTAX: unknown token 'X' at line N |
| Identifier not in ST | main.c         | print_token_stream() | [!] SEMANTIC: 'x' undeclared at line N  |

## 4. Results and Output

### 4.1 Compilation Pipeline Output

The compiler produces a 7-step sequential output when run against test.c. Each step is clearly demarcated with headers.

#### Step 1: Token Table

The lexer successfully tokenizes the source file into a structured table. For the test.c input, the compiler generates tokens covering all keywords, identifiers, operators, numbers, strings, and punctuation.

#### Sample Token Table Output:

| STEP 1: LEXICAL ANALYSIS - TOKEN TABLE |            |             |             |      |
|----------------------------------------|------------|-------------|-------------|------|
| No.                                    | TOKEN NAME | TOKEN VALUE | CATEGORY    | LINE |
| 1                                      | struct     | struct      | KEYWORD     | 6    |
| 2                                      | ID         | Point       | IDENTIFIER  | 6    |
| 3                                      | {          | {           | PUNCTUATION | 6    |
| 4                                      | int        | int         | KEYWORD     | 7    |
| 5                                      | ID         | x           | IDENTIFIER  | 7    |
| 6                                      | ;          | ;           | PUNCTUATION | 7    |
| 7                                      | int        | int         | KEYWORD     | 8    |
| 8                                      | ID         | y           | IDENTIFIER  | 8    |
| 9                                      | ;          | ;           | PUNCTUATION | 8    |
| 10                                     | }          | }           | PUNCTUATION | 9    |
| 11                                     | ;          | ;           | PUNCTUATION | 9    |
| 12                                     | int        | int         | KEYWORD     | 11   |
| 13                                     | ID         | arr         | IDENTIFIER  | 11   |
| 14                                     | [          | [           | PUNCTUATION | 11   |
| 15                                     | NUMBER     | 5           | NUMBER      | 11   |
| 16                                     | ]          | ]           | PUNCTUATION | 11   |
| 17                                     | ;          | ;           | PUNCTUATION | 11   |
| 18                                     | int        | int         | KEYWORD     | 12   |
| 19                                     | *          | *           | OPERATOR    | 12   |
| 20                                     | ID         | ptr         | IDENTIFIER  | 12   |
| 21                                     | ;          | ;           | PUNCTUATION | 12   |
| 22                                     | int        | int         | KEYWORD     | 14   |
| 23                                     | ID         | add         | IDENTIFIER  | 14   |
| 24                                     | (          | (           | PUNCTUATION | 14   |
| 25                                     | int        | int         | KEYWORD     | 14   |
| 26                                     | ID         | a           | IDENTIFIER  | 14   |
| 27                                     | ,          | ,           | PUNCTUATION | 14   |
| 28                                     | int        | int         | KEYWORD     | 14   |
| 29                                     | ID         | b           | IDENTIFIER  | 14   |
| 30                                     | )          | )           | PUNCTUATION | 14   |
| Token Category Summary:                |            |             |             |      |
| KEYWORD                                |            | :           | 33          |      |
| IDENTIFIER                             |            | :           | 61          |      |
| NUMBER                                 |            | :           | 19          |      |
| OPERATOR                               |            | :           | 35          |      |
| PUNCTUATION                            |            | :           | 84          |      |
| STRING                                 |            | :           | 1           |      |
| UNKNOWN                                |            | :           | 0           |      |
| -----                                  |            |             |             |      |
| TOTAL                                  |            | :           | 233         |      |

#### Step 2: Symbol Table

All declared variables, arrays, pointers, and functions are recorded with their types, scopes, and attributes.

#### Sample Symbol Table Output:

```
=====
STEP 2: SYMBOL TABLE
=====
```

| NAME   | TYPE   | KIND     | SCOPE  | LINE | INIT | SIZE | PARAMS |
|--------|--------|----------|--------|------|------|------|--------|
| x      | int    | Variable | global | 7    | YES  | -    | -      |
| y      | int    | Variable | global | 8    | NO   | -    | -      |
| Point  | struct | Variable | global | 9    | NO   | -    | -      |
| arr    | int    | Array    | global | 11   | NO   | 5    | -      |
| ptr    | int    | Pointer  | global | 12   | YES  | -    | -      |
| a      | int    | Variable | global | 14   | NO   | -    | -      |
| b      | int    | Variable | global | 14   | NO   | -    | -      |
| add    | int    | Function | global | 14   | NO   | -    | 2      |
| n      | int    | Variable | global | 18   | NO   | -    | -      |
| square | int    | Function | global | 18   | NO   | -    | 1      |
| main   | int    | Function | global | 22   | NO   | -    | 0      |
| z      | int    | Variable | main   | 26   | YES  | -    | -      |
| c      | int    | Variable | main   | 42   | YES  | -    | -      |
| i      | int    | Variable | main   | 59   | NO   | -    | -      |

```
-----
Total: 14 | Functions:3 Variables:9 Arrays:1 Pointers:1
=====
```

Step 3: Syntax & Semantic Analysis

Each token is analyzed for syntactic correctness, and identifiers are cross-referenced against the symbol table for semantic validation.

Sample Output:

```
=====
STEP 3: SYNTAX & SEMANTIC ANALYSIS
=====
Input File: test.c
```

|          |   |             |        |          |
|----------|---|-------------|--------|----------|
| Token 1  | : | KEYWORD     | struct | Line: 6  |
| Token 2  | : | IDENTIFIER  | Point  | Line: 6  |
| Token 3  | : | PUNCTUATION | {      | Line: 6  |
| Token 4  | : | KEYWORD     | int    | Line: 7  |
| Token 5  | : | IDENTIFIER  | x      | Line: 7  |
| Token 6  | : | PUNCTUATION | ;      | Line: 7  |
| Token 7  | : | KEYWORD     | int    | Line: 8  |
| Token 8  | : | IDENTIFIER  | y      | Line: 8  |
| Token 9  | : | PUNCTUATION | ;      | Line: 8  |
| Token 10 | : | PUNCTUATION | }      | Line: 9  |
| Token 11 | : | PUNCTUATION | ;      | Line: 9  |
| Token 12 | : | KEYWORD     | int    | Line: 11 |
| Token 13 | : | IDENTIFIER  | arr    | Line: 11 |
| Token 14 | : | PUNCTUATION | [      | Line: 11 |
| Token 15 | : | NUMBER      | 5      | Line: 11 |
| Token 16 | : | PUNCTUATION | ]      | Line: 11 |
| Token 17 | : | PUNCTUATION | ;      | Line: 11 |
| Token 18 | : | KEYWORD     | int    | Line: 12 |
| Token 19 | : | OPERATOR    | *      | Line: 12 |
| Token 20 | : | IDENTIFIER  | ptr    | Line: 12 |

Step 4: Abstract Syntax Tree

The AST is printed as a hierarchical text tree showing the program structure.

Sample AST Output:

```

=====
STEP 4: ABSTRACT SYNTAX TREE
=====

|-- Function: main
|-- return z
|   |-- ID: z
|-- printf("Result: %d")
|   |-- ID: z
|-- switch
|   |-- ID: z
|   |-- default
|       |-- = (x)
|           |-- L-- NUM: 0
|           |-- break
|   |-- case 1
|       |-- = (x)
|           |-- L-- NUM: 100
|           |-- break
|   |-- case 2
|       |-- = (x)
|           |-- L-- NUM: 200
|           |-- break
|-- for
|   |-- <
|       |-- ID: i
|       |-- L-- NUM: 5
|   |-- arr[i]= (arr)
|       |-- ID: i
|       |-- *
|           |-- ID: i
|           |-- L-- NUM: 2

```

### Step 5: Three Address Code (TAC)

TAC is generated for all expressions, assignments, control flow, and function calls.

### Sample TAC Output:

```

=====
STEP 5: THREE ADDRESS CODE (TAC)
=====

Tokenized form:
<struct>, <id1,Point>, <int>, <id2,x>, <int>, <id3,y>,
add>, <(>, <int>, <id7,a>, <,>, <int>, <id8,b>, <)>, <ret
<)>, <return>, <id10,n>, <*>, <id10,n>, <int>, <id11,mair
>, <int>, <id12,z>, <id4,arr>, <[>, <num,0>, <]>, <=>, <r
r>, <=>, <*>, <id2,x>, <id12,z>, <=>, <*>, <id5,ptr>, <id
<=>, <num,1>, <int>, <id8,b>, <=>, <num,0>, <int>, <id13,
<id8,b>, <if>, <(>, <id2,x>, <>>, <id3,y>, <)>, <id12,z>,
x>, <while>, <(>, <id2,x>, <>>, <num,0>, <)>, <id2,x>, <=
14,i>, <<>, <num,5>, <id14,i>, <++>, <)>, <id4,arr>, <[>,
>, <case>, <num,1>, <:>, <id2,x>, <=>, <num,100>, <break>,
, <id2,x>, <=>, <num,0>, <break>, <printf>, <(>, <str,"Re

Three Address Code:
-----

1: param int a
1: param int b

>>> Function: add <<<
1: t0 = id2(a) + id3(b)
2: return t0
>>> End: add <<<

3: param int n

>>> Function: square <<<
3: t1 = id5(n) * id5(n)
4: return t1
>>> End: square <<<

```

## Step 6: Optimized TAC

After applying Copy Propagation and Constant Folding, redundant temporaries are eliminated and constant expressions are pre-evaluated.

### Sample Optimized TAC Output:

```
=====
STEP 6: CODE OPTIMIZATION - Optimized TAC
=====
Optimizations: [1] Copy Propagation [2] Constant Folding

Optimized Three Address Code:
-----
1: param int a
1: param int b

>>> Function: add <<<
1: t0 = id2(a) + id3(b)
2: return t0
>>> End: add <<<

3: param int n

>>> Function: square <<<
3: t1 = id5(n) * id5(n)
4: return t1
>>> End: square <<<
```

## Step 7: Assembly Code Generation

x86 NASM-style assembly is generated from the optimized TAC.

### Sample Assembly Output:

```
=====
STEP 7: ASSEMBLY CODE GENERATION (x86 NASM style)
=====

add:
    push ebp
    mov  ebp, esp
    mov  eax, [a]
    add  eax, [b]
    mov  [t0], eax
    mov  eax, [t0]
    pop  ebp
    ret
    pop  ebp
    ret
    ; param n

square:
    push ebp
    mov  ebp, esp
    mov  eax, [n]
    imul eax, [n]
    mov  [t1], eax
    mov  eax, [t1]
    pop  ebp
    ret
    pop  ebp
    ret
```

## 4.2 Error Detection Output Examples

The following examples show what the compiler outputs when it encounters different types of errors.

### 4.2.1 Scenario 1: Clean Input (test.c — No Errors)

When the provided test.c is compiled (which is valid C), the compiler reports:

```
Syntax Errors   : 0
Semantic Errors : 0
[OK] ANALYSIS PASSED
```

And the final summary shows:

```
=====
COMPILATION COMPLETE
=====

Total Tokens   : 195
Total Symbols  : 18
Total Errors   : 0
Total TAC      : 112
=====
```

### 4.2.2 Scenario 2: Lexical Error — Unknown Character

Input (modified test.c):

```
int main() {
    int x = @5; /* '@' is not a valid C character */
    return x;
}
```

Compiler Output:

```
[LEX ERROR] Unknown char '@' at line 2

=====
STEP 3: SYNTAX & SEMANTIC ANALYSIS
=====

...
Token 7   : UNKNOWN           | @                               | Line: 2
[!] SYNTAX: unknown token '@' at line 2
-----

Syntax Errors   : 1
Semantic Errors : 0
[FAIL] ANALYSIS FAILED
-----

=====
COMPILATION COMPLETE
=====

Total Errors   : 2
```

### 4.2.3 Scenario 3: Semantic Error — Undeclared Variable

Input:

```
int main() {
    x = 10;    /* 'x' was never declared */
    return x;
}
```

Compiler Output (during parsing):

```
[WARN] Line 2: 'x' undeclared
```

Step 3 Output:

```
Token 4 : IDENTIFIER      | x                               | Line: 2
[!] SEMANTIC: 'x' undeclared at line 2

Syntax Errors : 0
Semantic Errors: 1
[FAIL] ANALYSIS FAILED
```

#### 4.2.4 Scenario 4: Semantic Error — Redeclared Variable

Input:

```
int main() {
    int x = 5;
    int x = 10; /* 'x' declared again in same scope */
    return x;
}
```

### Compiler Output:

```
[WARN] Line 3: 'x' redeclared
```

Symbol Table shows only one entry for x:

| NAME                                                   | TYPE | KIND     | SCOPE  | LINE | INIT | SIZE | PARAMS |
|--------------------------------------------------------|------|----------|--------|------|------|------|--------|
| main                                                   | int  | Function | global | 1    | NO   | -    | 0      |
| x                                                      | int  | Variable | main   | 2    | YES  | -    | -      |
| Total: 2   Functions:1 Variables:1 Arrays:0 Pointers:0 |      |          |        |      |      |      |        |

# 5. Conclusion and Future Scope

## 5.1 Conclusion

This project successfully demonstrates a fully functional Mini C Compiler built from scratch using industry-standard tools: Flex for lexical analysis and Bison for parsing. The compiler correctly handles a significant subset of the C programming language and implements all major phases of compilation:

1. Lexical Analysis: Correctly tokenizes C source code including comments, operators, keywords, identifiers, numbers, and strings.
2. Symbol Table: Tracks all variables, arrays, pointers, functions, and struct definitions with their types, scopes, and initialization status.
3. Syntax & Semantic Analysis: Detects and reports redeclarations, undeclared variable usage, and syntax errors with line numbers.
4. AST Generation: Builds a complete Abstract Syntax Tree that accurately represents the program structure in a human-readable tree format.
5. TAC Generation: Generates correct Three Address Code with proper temporary variable allocation, label management, short-circuit evaluation for `&&` and `||`, and support for all control flow structures.
6. Code Optimization: Successfully applies Copy Propagation (eliminating redundant temporaries) and Constant Folding (pre-computing constant expressions).
7. Assembly Generation: Produces valid x86 NASM-style assembly code with correct `.data`, `.bss`, and `.text` sections, function prologues/epilogues, and instruction mapping for all operations.

The project demonstrates a strong understanding of compiler theory and its practical application. The modular design of the codebase makes each phase independently readable and verifiable.

## 5.2 Future Scope

While the current implementation covers a broad range of C features, several enhancements could be made in future versions:

### 5.2.1 Language Feature Extensions

| Feature                                                        | Current Status  | Future Plan           |
|----------------------------------------------------------------|-----------------|-----------------------|
| do-while loop                                                  | Not supported   | Add grammar rule      |
| typedef                                                        | Not supported   | Add to symbol table   |
| Multi-dimensional arrays                                       | Not supported   | Extend array handling |
| Struct member access ( <code>.</code> and <code>-&gt;</code> ) | Parsed (no TAC) | Full semantic support |
| enum and union                                                 | Not supported   | Add type system       |



|                         |               |                        |
|-------------------------|---------------|------------------------|
| #include / preprocessor | Not supported | Add preprocessor phase |
| String operations       | Limited       | Full string support    |
| Type casting            | Not supported | Add cast expressions   |

### 5.2.2 Compiler Improvements

1. **Better Semantic Analysis:** Full type checking for expressions, function call argument type matching, return type verification.
2. **Scope Management:** Implement proper block-level scoping using a scope stack, allowing local variables in nested blocks.
3. **Error Recovery:** Improve error recovery to continue parsing and report multiple errors in a single pass.
4. **More Optimization Passes:**
  - a. Dead Code Elimination
  - b. Common Subexpression Elimination
  - c. Loop Invariant Code Motion
  - d. Register Allocation (graph coloring)
5. **Real Code Generation:** Target actual NASM/GAS assembly that can be assembled and linked with nasm and ld or gcc.
6. **Intermediate Representation:** Switch to SSA (Static Single Assignment) form for more sophisticated optimization.
7. **Symbol Table Scoping:** Use a hash-table based symbol table for O(1) lookup instead of linear search.

### 5.2.3 Tooling Improvements

- **Graphical AST Viewer:** Generate DOT format output for visualization with Graphviz.
- **Error Message Quality:** Improve error messages with suggestions (similar to Clang's diagnostics).
- **IDE Integration:** Language server protocol (LSP) support for syntax highlighting and error reporting.
- **Testing Framework:** Automated test suite with multiple C programs to validate compiler correctness.

## 6. References

1. Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). Compilers: Principles, Techniques, and Tools (2nd ed.). Pearson/Addison-Wesley. (The "Dragon Book")
2. Appel, A. W., & Ginsburg, M. (2004). Modern Compiler Implementation in C. Cambridge University Press.
3. Cooper, K., & Torczon, L. (2011). Engineering a Compiler (2nd ed.). Morgan Kaufmann.
4. Flex (Fast Lexical Analyzer). (n.d.). Retrieved from <https://github.com/westes/flex>

5. GNU Bison — The YACC-compatible Parser Generator. (n.d.). Free Software Foundation. Retrieved from <https://www.gnu.org/software/bison/>
6. NASM — The Netwide Assembler. (n.d.). Retrieved from <https://www.nasm.us/>
7. Kernighan, B. W., & Ritchie, D. M. (1988). The C Programming Language (2nd ed.). Prentice Hall.
8. Levine, J. R. (2000). Flex & Bison: Text Processing Tools. O'Reilly Media.
9. GCC, the GNU Compiler Collection. (n.d.). Free Software Foundation. Retrieved from <https://gcc.gnu.org/>
10. Intel. (2021). Intel® 64 and IA-32 Architectures Software Developer's Manual. Intel Corporation. Retrieved from <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>

## 7. Appendices

Appendix A: Full Source Code (GitHub Link)

Github Link: <https://github.com/abdullahalmusabbir/Mini-Compiler-For-C>

Repository Structure:

|                                                                                                                                            |                                |                                                                                                      |
|--------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------|------------------------------------------------------------------------------------------------------|
|  <b>abdullahalmusabbir</b> <a href="#">add more token</a> | b90b790 · last week            |  <b>6 Commits</b> |
|  .gitignore                                              | 1st commit                     | last week                                                                                            |
|  22101039.pptx                                          | <a href="#">add more token</a> | last week                                                                                            |
|  Makefile                                               | <a href="#">add more token</a> | last week                                                                                            |
|  Report.docx                                            | <a href="#">Add Report</a>     | last week                                                                                            |
|  compiler.h                                             | <a href="#">add more token</a> | last week                                                                                            |
|  lexer.l                                                | <a href="#">add more token</a> | last week                                                                                            |
|  main.c                                                 | <a href="#">add more token</a> | last week                                                                                            |
|  parser.y                                               | <a href="#">add more token</a> | last week                                                                                            |
|  test.c                                                 | <a href="#">add more token</a> | last week                                                                                            |

Appendix B: Sample Input File (test.c)

```
/* multi-line comment — this will be ignored */
// single line comment
```

```
struct Point {
    int x;
    int y;
};
```

```
int arr[5];
```

```

int *ptr;

int add(int a, int b) {
    return a + b;
}

int square(int n) {
    return n * n;
}

int main() {
    /* variables */
    int x = 10;
    int y = 20;
    int z;

    /* array */
    arr[0] = 5;
    arr[1] = x + 3;

    /* pointer */
    ptr = &x;
    z = *ptr;

    /* function call */
    z = add(x, y);

    /* && and || TAC */
    int a = 1;
    int b = 0;
    int c;
    c = a && b;
    c = a || b;

    /* if-else */
    if (x > y) {
        z = x - y;
    } else {
        z = y - x;
    }

    /* while */
    while (x > 0) {
        x = x - 1;
    }

    /* for loop */
    for (int i = 0; i < 5; i++) {

```

```

    arr[i] = i * 2;
}

/* switch-case */
switch (z) {
    case 1:
        x = 100;
        break;
    case 2:
        x = 200;
        break;
    default:
        x = 0;
        break;
}

printf("Result: %d", z);
return z;
}

```

#### Appendix C: Error Types and Handling

| Error Type            | Prefix        | Example                         |
|-----------------------|---------------|---------------------------------|
| Lexical Error         | [LEX ERROR]   | Unknown character @ at line 5   |
| Parse Error           | [PARSE ERROR] | Unexpected token at line 12     |
| Semantic Warning      | [WARN]        | Variable x undeclared at line 8 |
| Redeclaration Warning | [WARN]        | y redeclared at line 15         |